Capítulo 3: Gestión del almacenamiento

Introducción



Tabla de contenidos

- Bloque 1: SGBD relacional 4
- Bloque 2: Uso de memoria principal y disco 8
- Bloque 3: Almacenamiento de las tablas 11
- Bloque 4: Índices 14
- Bloque 5: Otras informaciones a almacenar 22
- Bloque 6: El catálogo o diccionario de datos 31
- Bloque 7: El gestor de almacenamiento 37

Objetivos de aprendizaje

- Comprender los beneficios de estudiar el funcionamiento de los sistemas gestores de bases de datos (SGBD) relacionales.
- Poder deducir las necesidades de almacenamiento de un SGBD relacional.
- Poder deducir la necesidad de un gestor de almacenamiento y analizar sus responsabilidades.
- Comprender los conceptos asociados al almacenamiento de un SGBD relacional.

Bloque 1: SGBD relacional
Del diseño a la implementación

- Para implementar un diseño relacional y consultas en álgebra relacional utilizaremos SQL (lo harán en el taller de la materia).
- SQL permite: definir esquemas; definir restricciones de integridad, insertar, modificar y eliminar datos; expresar consultas.
- Sin embargo, por si solo SQL no ejecuta nada.
- Necesita un sistema capaz de interpretar esas instrucciones.
- Un **SGBD relacional** recibe instrucciones SQL y se encarga de verificar restricciones de integridad; almacenar los datos; procesar las consultas; optimizar su ejecución; devolver los resultados al usuario.
- Estudiaremos como un SGBD realiza estas tareas.

Bloque 1: SGBD relacional
Por qué estudiar el funcionamiento de un SGBD relacional

- **Comprender cómo funciona internamente un SGBD relacional permite:**
  - Escribir consultas más eficientes.
  - Comprender por qué algunas consultas son más costosas que otras.
  - Entender las decisiones que toma el optimizador.
  - Diseñar mejores bases de datos.
- Pero además aparecen conceptos que aparecen en muchos otros sistemas de gestión de información.

Bloque 1: SGBD relacional
Por qué estudiar el funcionamiento de un SGBD relacional

- Muchos sistemas actuales organizan la información de manera diferente.
- **Por ejemplo**: bases de datos basadas en documentos, bases de datos de grafos, sistemas de recuperación de la información.
- Aunque utilizan modelos de datos distintos, **todos deben realizar tareas similares**:
  - almacenar información, procesar consultas, optimizar su ejecución; devolver resultados al usuario.
- Por ello, comprender el funcionamiento de un SGBD relacional constituye una excelente base para estudiar otros paradigmas de gestión de la información.

Bloque 1: SGBD relacional
¿Qué debe almacenar un SGBD?

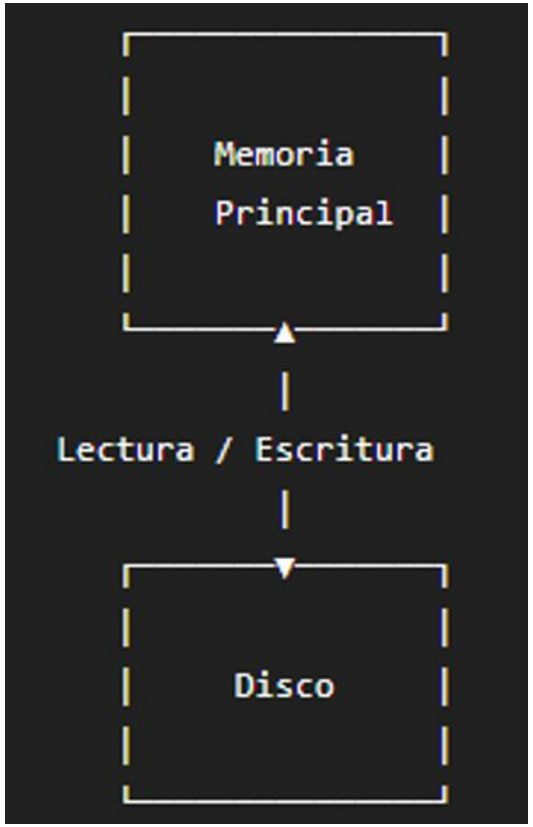
- Hasta aquí hemos visto cómo encarar el diseño de una base de datos y esa es la primera mitad de la materia.
- La segunda mitad es sobre sistemas de bases de datos, comenzando con los SGBD relacionales.
- Un SGBD no administra solo los datos de los usuarios.
  - También debe administrar la propia definición de la bases de datos.
- Entonces un SGBD relacional almacena dos tipos de información:
  - Los datos de la organización o usuario (tuplas de tablas).
  - La descripción de la base de datos: esquema relacional y restricciones de integridad.
- Los datos cambian continuamente, pero la descripción cambia con menor frecuencia.

Bloque 2: Uso de memoria principal y disco
Contexto

- La memoria rápida (p.ej. memoria RAM) es finita y pierde los datos al apagar el sistema.
- La memoria permanente (p.ej. disco rígido, SSD) es lenta, mecánica o de bloques, pero sobrevive al fallo eléctrico.
- Hay informaciones que queremos que permanezcan (p.ej. las tablas de la BD).
- Existen necesidad de procesar ciertos datos (p.ej. de las tablas).

Bloque 2: Uso de memoria principal y disco ¿Cuándo programaban en algorítmica, donde se procesaba la información?

- Los programas que construían trabajan de la siguiente manera:
  - Las informaciones que queremos que **permanezcan** se **guardan en disco** para poder recordarlas.
  - Cuando un programa quiere usar esas informaciones en disco, las **carga en memoria**.
  - Todas las operaciones sobre esas informaciones se hacen en memoria, porque es mucho más rápida que el disco.
  - Si los resultados de operar queremos preservarlos entonces los almacenamos en disco.
  - Si al operar obtenemos modificaciones en las informaciones en los archivos y queremos conservar esos cambios, estas alteraciones se escriben nuevamente en disco.



Bloque 2: Uso de memoria principal y disco
El mismo principio en un SGBD

- **Un SGBD sigue la misma idea. ¿Cómo se la aplica?**
  - Los datos que queremos que persistan permanecen almacenados en disco.
  - Por ejemplo, las tablas de la base de datos.
- Cuando una consulta necesita acceder a ellos o queremos alterarlos, el sistema los **trae a memoria**.
- Para el caso de alteraciones a los datos en memoria, los cambios se escriben nuevamente en disco.

Bloque 3: Almacenamiento de las tablas
¿Cómo conservamos información en un programa?

- En programación vieron que:
  - Hay programas que acceden a los datos y los procesan.
  - Muchos de esos programas producen **información nueva** que debe sobrevivir a su ejecución.
  - Por lo tanto, dicha información no puede permanecer solamente en memoria.
- Por eso utilizamos: archivos y estructuras de datos almacenados en archivos.
  - De esta manera la información permanece disponible aunque el programa finalice.

Bloque 3: Almacenamiento de las tablas

- **Asunto**: Una base de datos debe poder almacenar los datos de manera persistente y organizada.
- **Problema**: como organizamos y almacenamos persistentemente los datos
- **Solución previa a los SGBD**: se almacenaban los datos usando archivos planos.
  - En algoritmos normalmente uno piensa: archivo → estructuras de datos en disco.
  - Los datos se guardaban en texto o binario, sin relaciones entre registros.
  - Por ejemplo, listas secuenciales.
  - La estructura de estos archivos era responsabilidad del programador.
  - Si cambiaba la estructura de un archivo de datos, había que modificar todos los programas que lo accedían o alteraban.

Bloque 3: Almacenamiento de las tablas
Diferencia entre las bases de datos y los algoritmos

- **Solución de los SGBD relacionales**: un SGBD almacenará las tablas usando archivos
  - Se utilizan **estructuras de almacenamiento en disco** para poder acceder y almacenar esos registros.
  - Normalmente una tabla ocupa varios **bloques** y un bloque tiene una cierta cantidad de **registros** de la tabla.
  - En SGBD relacionales se piensa: una tabla → archivo → estructura de datos.
  - O sea que en SGBD hay dos niveles en lugar de uno:
    - **lógico** (tabla) y **físico** (estructura de datos)
  - El usuario solo define las tablas y no tiene que preocuparse por mantener programas que las acceden.
  - Las estructuras de datos en disco para las tablas son creadas y mantenidas automáticamente por el SGBD.
  - La estructura interna en disco que usa la tabla **determinará** el **costo de acceder a los registros**.

Bloque 4: Índices
Contexto

- **Asunto**: Acceso rápido a los datos.
  - Un archivo de datos puede ser muy grande
  - Por lo tanto, recorrer un archivo de datos secuencialmente para encontrar una información puede ser muy costoso.
  - Hace falta un mecanismo para acceder rápido a los datos.
- **Problema:** ¿Cómo **encontrar rápidamente** un registro dentro de un archivo **muy grande**?

Bloque 4: Índices
¿Cómo buscar rápidamente información?

- Supongamos que queremos encontrar todas las páginas donde aparece el término ‘normalización’ en un libro de 800 páginas. **¿Qué harían?**

Bloque 4: Índices
¿Cómo buscar rápidamente información?

- Supongamos que queremos encontrar todas las páginas donde aparece el término ‘normalización’ en un libro de 800 páginas. **¿Qué harían?**
- Buscar en el índice atrás del libro.
- **¿Por qué existe un índice?**

Bloque 4: Índices
¿Cómo buscar rápidamente información?

- Supongamos que queremos encontrar todas las páginas donde aparece el término ‘normalización’ en un libro de 800 páginas. **¿Qué harían?**
- Buscar en el índice atrás del libro.
- **¿Por qué existe un índice?**
- Sin índice habría que recorrer página por página.
- El índice almacena únicamente una parte de la información.
  - P.ej. normalización … 315, dependencia funcional … 287.
- El índice solo indica dónde encontrar la palabra.

Bloque 4: Índices

- **Solución previa a los SGBD**: ISAM y VSAM en COBOL.
  - **ISAM**: **Método de acceso secuencial indexado**.
    - Los registros se almacenaban en orden secuencial.
    - Se construía un índice que permitía localizar más rápido un registro por clave.
    - El programador podía acceder tanto secuencialmente como directamente por clave usando el índice.
  - **VSAM (Virtual Storage Access Method)**
    - fue la evolución de ISAM en sistemas IBM.
    - Permitía manejar **datasets** más grandes y flexibles, con acceso secuencial, indexado y directo.
    - El programador definía explícitamente el archivo, la clave y el modo de acceso.

Bloque 4: Índices

- **Solución previa a los SGBD**: ISAM y VSAM en COBOL.
  - Aunque ISAM/VSAM ofrecían índices y acceso más rápido, la **lógica de uso estaba embebida en el código COBOL.**
  - El **programador debía definir** cómo abrir el archivo, cómo recorrerlo, cómo usar el índice.
  - El programador debía crear y mantener los índices como parte de la lógica de los programas.
  - **Ejemplo**: al insertar un registro en un archivo ISAM/VSAM, el programa debía actualizar la estructura del índice (árbol o tabla de claves).
  - Si se borraba o modificaba un registro, también había que ajustar manualmente el índice.
  - En otras palabras, el índice era una **estructura física** que el programa debía manipular directamente.
  - Esto hacía que la integridad y eficiencia dependieran del código escrito en COBOL.

Bloque 4: Índices

- **Solución de SGBD relacional:** Un SGBD relacional permite definir índices sobre las tablas.
  - Muchas consultas buscan objetos a partir de ciertos atributos.
    - P.ej. SELECT * FROM Alumno WHERE legajo = 1025
    - Podríamos recorrer toda la tabla Alumno o bien consultar un índice de legajos.
    - El índice va a indicar dónde se encuentra el registro buscado.
  - Entonces los **índices** no almacenan toda la información.
    - Almacenan únicamente ciertos valores de búsqueda y para cada uno da una o más referencias hacia los registros correspondientes.
    - El SGBD usa esa referencia para acceder rápidamente al registro completo.

Bloque 4: Índices

- **Solución de SGBD relacional:**
  - En SQL un usuario declara un índice con una sentencia como:
    - CREATE INDEX idx_cliente ON clientes(id_cliente)
  - A partir de ahí, el **DBMS se encarga automáticamente** de:
    - Crear la estructura física del índice.
    - Mantenerlo actualizado cada vez que se inserta, modifica o borra un registro en la tabla.
    - Decidir cuándo usarlo en una consulta.
- **Ganancia de los SGBD relacionales**:
  - El programador ya no necesita escribir rutinas para actualizar índices: el sistema lo hace de forma transparente.

Bloque 5: Otras informaciones a almacenar

- **Asunto**: Almacenar otras cosas que hagan falta además de los datos.
- **Antes de los SBGD**: lo que se almacenaba eran solo **datos en archivos** y, en algunos casos, **estructuras de índice** asociadas para acelerar el acceso.
- Hasta ahora vimos que las tablas necesitan almacenarse. Cada tabla puede implementarse mediante uno o más archivos en disco.
- Pero surge una pregunta: **¿alcanza con almacenar únicamente los datos e índices?**

Bloque 5: Otras informaciones a almacenar

- En los SGBD relacionales además de los datos de una tabla existen otras informaciones asociadas a la tabla:
  - **Información sobre el esquema de la BD**: Qué tabla representan, qué atributos posee esta tabla; qué tipo tiene cada atributo, qué restricciones deben cumplirse.
  - Qué índices sobre tablas existen.
- **Problema**: ¿Cómo almacenar el esquema de la base de datos?

Bloque 5: Otras informaciones a almacenar

- **Antes de los SGBD**: No se almacenaba el esquema de los datos. El esquema se codificaba en los programas:
  - La definición de los campos (longitud, tipo, orden) se escribía directamente en el código COBOL.
  - Cada aplicación debía conocer la estructura del archivo.
  - Si se cambiaba el formato, había que modificar todos los programas que lo usaban.
  - Unicidad de claves, rangos válidos, relaciones entre archivos se codificaban como rutinas en el programa.
  - **Ejemplo**: antes de insertar un cliente, el programa debía recorrer el archivo para verificar que el código no existiera.
  - No había enforcement automático: si un programa olvidaba validar, se podían almacenar datos inválidos.
  - **En conclusión**:
    - El esquema de la BD está embebido en cada programa.
    - Las restricciones de integridad se validan manualmente en código.
    - La confiabilidad de la BD depende de la disciplina del programador.

Bloque 5: Otras informaciones a almacenar
Contexto de los SGBD

- Cuando se recibe una consulta SQL como

  SELECT Nombre FROM Alumno WHERE Legajo = 1234;

- **Para procesar la consulta el SGBD debe responder preguntas como**:
  - ¿Existe una tabla llamada alumno?
  - ¿Pertenece a la base de datos?
  - ¿Existe un atributo llamado Nombre?
  - ¿Qué tipo tiene Legajo? (no sea que haya un error)
  - ¿Existe un índice sobre legajo? (para poder responder más rápido a la consulta)
- **Toda esta información es necesaria antes de acceder a los datos.**

Bloque 5: Otras informaciones a almacenar

- Los SGBD relacionales hacen una **separación completa y absoluta** entre los datos y todo lo demás
  - p.ej. el esquema de la BD, los índices, tipos de datos definidos por el usuario, etc.
  - por lo tanto, los datos **no contienen información** acerca de atributos, tipos de datos y restricciones sobre los atributos.

Bloque 5: Otras informaciones a almacenar

- Toda la información que necesita el SGBD relacional para poder procesar las consultas necesita estar almacenada.
- Los registros almacenados en los archivos de las tablas no contienen información como:
  - nombres de atributos,
  - tipos de datos,
  - claves primarias,
  - claves foráneas,
  - índices,
  - restricciones de integridad.

Bloque 5: Otras informaciones a almacenar

- **Solución de los SGBD relacionales**: Toda esta información de los esquemas y declaración de índices debe almacenarse por separado.
  - El usuario la declara a esta información en SQL.
  - El usuario no se tiene que preocupar por escribir programas que chequean las restricciones de integridad.
    - Esto lo hace el SGBD automáticamente.
    - **Por ejemplo**, cuando hay que alterar tablas.
    - La confiabilidad de la BD está garantizada por el sistema.

Bloque 5: Otras informaciones a almacenar Entonces necesitamos una estructura adecuada

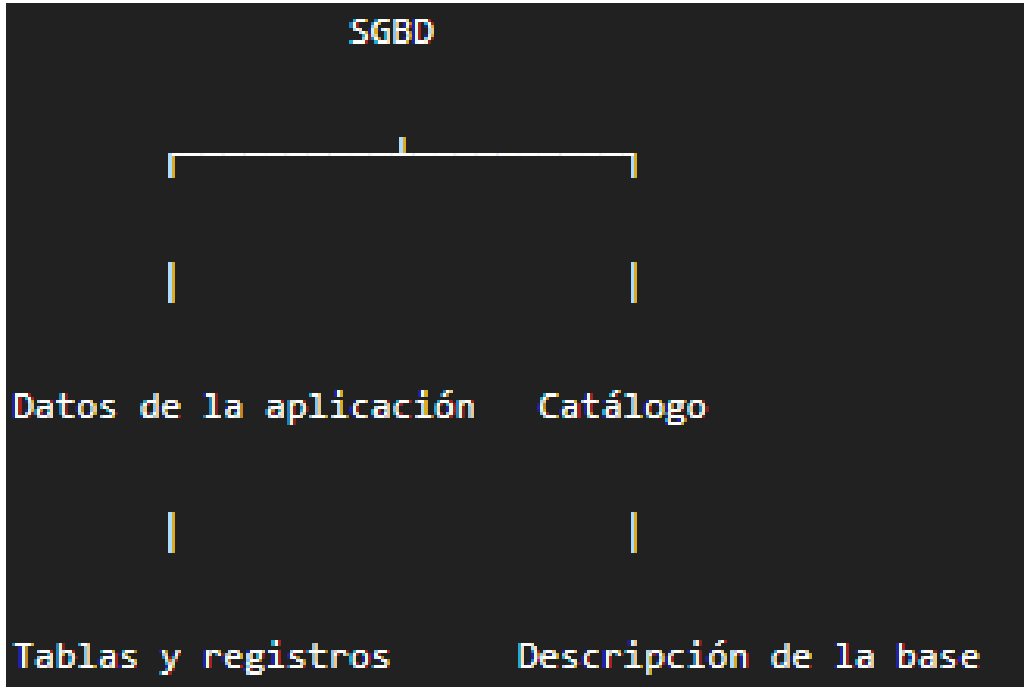
- **Problema**: Para darle forma a las otras informaciones a almacenar,
  - **¿qué tipo de estructura conviene que tenga?**
- El SGBD necesita almacenar información sobre la propia BD.
- **Por ejemplo:**
  - Alumno: tabla, nombre de archivo.
  - Legajo: atributo, tipo entero.
  - IDX_ALUMNO_LEGAJO: índice, nombre de archivo.
  - CP_ALUMNO: clave primaria, tabla, atributos.
  - CF_CURSA: clave foránea, tabla curso, Legajo, tabla Alumno.

Bloque 5: Otras informaciones a almacenar Entonces necesitamos una estructura adecuada

- **Solución**: Para cada objeto del esquema, el sistema debe conocer la información necesaria para poder administrarlo.
  - nombre de objeto → información necesaria para administrarlo.
- Esto es lo que hacen **los diccionarios**: responde preguntas sobre sus palabras: ¿Existe la palabra?¿Que función gramatical tiene?¿Qué significa?
  - Es decir, tanto la información de la base de datos como un diccionario cumplen la misma función:
  - asocian un nombre con la información necesaria para interpretar correctamente ese nombre.

Bloque 6: El catálogo o diccionario de datos

- Toda la información sobre los datos que el SGBD necesita para hacer consultas, y alteraciones a la BD es un **sistema de nombres** y se almacena en una estructura especial denominada **diccionario de datos** (o **catálogo del sistema**).
- El catálogo almacena información como: tablas, atributos, tipos de datos de atributos, restricciones, índices, vistas y demás objetos definidos por el usuario.



Bloque 6: El catálogo o diccionario de datos ¿Cómo se almacena el catálogo?

- El catálogo también necesita **persistir** cuando el sistema se apaga.
- **Problema**: ¿Cómo almacenar el catálogo?
- **Solución**: Muchos SGBD almacenan el catálogo usando los mismos mecanismos generales usados para almacenar las tablas de la aplicación.
  - Conceptualmente el catálogo también se guarda en disco mediante **archivos** y **estructuras de almacenamiento** administradas por el SGBD.
  - En muchos sistemas reales, el catálogo está organizado como un conjunto de **tablas especiales** del sistema.

Bloque 6: El catálogo o diccionario de datos
¿Para qué usa un SGBD el catálogo?

- El SGBD usa el catálogo constantemente para:
  - validar consultas,
  - encontrar índices,
  - verificar restricciones de integridad antes de hacer alteraciones de datos.
- El catálogo es una pieza central en prácticamente toda operación que hace el SGBD.

Bloque 6: El catálogo o diccionario de datos ¿Cómo accedemos a la información del catálogo?

- Hasta ahora sabemos que el catálogo almacena la descripción de la base de datos.
- **Problema:** ¿Cómo accedemos la información del catálogo?
- **Por ejemplo**: ¿Cómo averiguar qué tablas existen? ¿qué atributos tiene una tabla? ¿qué índices están definidos para una tabla? ¿Cuáles son las claves primarias? ¿Cuáles son las claves foráneas?
- **Solución**: Muchos SGBD permiten consultar el catálogo usando SQL.
- **Por ejemplo**:
  - SELECT * FROM INFORMATION_SCHEMA.TABLES;
  - SELECT * FROM INFORMATION_SCHEMA.COLUMNS;
- Es decir, el catálogo puede también consultarse.

Bloque 6: El catálogo o diccionario de datos ¿Cómo modificamos el catálogo?

- El catálogo no solamente puede consultarse.
- También puede **modificarse**.
- **Problema**: ¿Cómo modificamos el catálogo?
- **Solución**: Cada vez que ejecutamos instrucciones de SQL como:
  - CREATE TABLE (agregar definición de tabla)
  - ALTER TABLE (modificar esquema de tabla en el catálogo)
  - DROP TABLE (borrar definición de tabla)
  - CREATE INDEX (registra el índice en el catálogo)
- el SGBD modifica el catálogo. Modifica la descripción de la base de datos.
- **Conclusión**: el catálogo termina comportándose **casi como otra base de datos**.

Bloque 6: El catálogo o diccionario de datos Yendo un poco más profundo

- Se desprende de las operaciones anteriores que el SGBD debe mantener una correspondencia entre el catálogo y los objetos físicos asociados.
  - Cada vez que cambia uno, debe cambiar el otro.

Instrucción	Catálogo	Estructuras físicas
CREATE TABLE	Agrega la definición de la tabla	Crea el almacenamiento para la tabla
ALTER TABLE	Modifica la definición	Puede reorganizar los datos
DROP TABLE	Elimina la definición	Elimina el almacenamiento
CREATE INDEX	Registra el índice	Construye el índice

Bloque 7: El gestor de almacenamiento

- **Asunto**: hace falta gestionar el almacenamiento de la base de datos.
- **Problema**: ¿cómo gestionar el almacenamiento de los datos?
- **Solución antes de los SGBD**: La gestión del almacenamiento estaba **dispersa en los programas** y dependía de rutinas específicas escritas por los programadores.
  - Los datos se guardaban en archivos secuenciales.
  - Cada aplicación debía abrir, leer, escribir y cerrar archivos usando llamadas al sistema operativo.
  - No había abstracción: el programa debía conocer la ubicación física y el formato exacto.
  - En ISAM/VSAM se añadían estructuras de índice para acelerar búsquedas.
  - El programador debía mantener esos índices al insertar, borrar o modificar registros.
  - La definición de campos y registros estaba en el código COBOL.
  - No había un catálogo central: cada programa repetía la definición.

Bloque 7: El gestor de almacenamiento
Contexto para SGBD

- Hasta ahora sabemos que:
  - las tablas se almacenan en archivos;
  - los índices también se almacenan en archivos;
  - el catálogo se almacena en archivos;
  - las consultas trabajan sobre datos en memoria.
- Hay consultas sobre archivos.
- Existe la necesidad de alterar los archivos
- Hace falta leer y escribir archivos.

Bloque 7: El gestor de almacenamiento ¿Quién accede a los archivos?

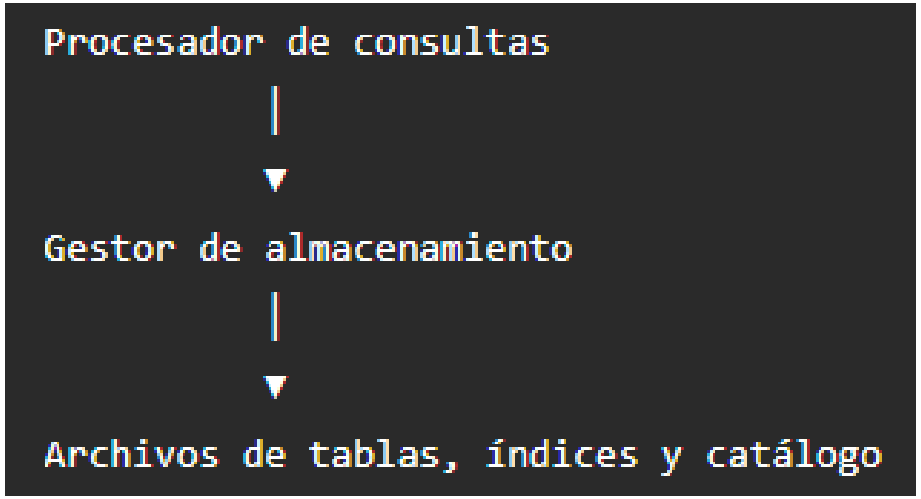
- Surge una pregunta: **¿Debe cada componente usada saber cómo leer y escribir archivos?**
- **Principio de diseño**: Cuando muchas funciones o procedimientos necesitan realizar la misma tarea, **conviene concentrarla en un único módulo** especializado.
- **Antes de los SGBD**: no había un gestor central de almacenamiento. Las consecuencias de su carencia son:
  - **Código repetido**: cada aplicación debía implementar rutinas de acceso, validación y actualización.
  - **Dependencia absoluta del programa**: si cambiaba el formato del archivo, había que modificar todos los programas.
  - **Falta de independencia lógica/física**: los datos estaban atados a cómo los programas los manejaban.
  - **Integridad dispersa**: las validaciones se codificaban manualmente en cada aplicación, sin enforcement automático.

Bloque 7: El gestor de almacenamiento

- En este caso, varios de los componentes del SGBD necesitan acceder a: archivos de tablas; archivos de índices; bloques en disco.
- **Solución de los SGBD relacionales**: Se construye un **componente especializado en administrar el almacenamiento**.
  - El **gestor de almacenamiento** administra el almacenamiento físico de la base de datos.

Bloque 7: El gestor de almacenamiento

- Los demás componentes del SGBD no manipulan directamente los archivos.
  - En su lugar, **solicitan servicios** al gestor de almacenamiento.
  - En otras palabras, el gestor de almacenamiento provee una interfaz entre
  - el almacenamiento físico y las consultas y actualizaciones.



Bloque 7: El gestor de almacenamiento
¿De qué debe ocuparse el gestor de almacenamiento?

- **El gestor de almacenamiento es responsable de:**
  - Administrar los archivos que almacenan tablas, índices, y catálogo;
  - Organizar la información en archivos (p.ej. bloques, registros, etc.)
  - Localizar datos dentro de archivos e índices.
  - Mover bloques entre disco y memoria.
  - Incorporar, modificar, y eliminar información almacenada.
  - Mantener sincronizado el contenido de memoria con el contenido persistente.

Bloque 7: El gestor de almacenamiento
¿Qué operaciones debe ofrecer el gestor de almacenamiento?

- A partir de las responsabilidades del gestor de almacenamiento podemos deducir las operaciones básicas que necesita ofrecer.
- Estas operaciones son **primitivas de bajo nivel**.
  - Por si solas no permiten responder una consulta completa,
  - pero proporcionan **servicios** sobre los que trabajará el procesamiento de consultas.

Responsabilidad	Operaciones posibles
Administrar archivos	crear, abrir, cerrar y eliminar archivos
Mover información	leer un bloque a memoria, escribir un bloque en disco
Localizar información	buscar una entrada en un índice, recorrer un archivo
Administrar registros	leer, insertar, modificar y eliminar registros
Mantener la persistencia	registrar cambios y escribir bloques modificados

Bloque 7: El gestor de almacenamiento

- **Aclaración importante**: se trata que las primitivas del gestor de almacenamiento son independientes del tipo de objeto
  - al menos para las responsabilidades de mover información, administrar registros y mantener la persistencia.
  - Es decir en lugar de administrar objetos como tablas, catálogos o índices, administran archivos y registros.
- Las operaciones de la responsabilidad **administrar registros almacenados** pueden actuar sobre registros pertenecientes a tablas, índices o tablas del catálogo.

Índice de términos

- Bloques: 13
- Catálogo del sistema: 31
- Diccionario de datos: 31
- Estructura de almacenamiento en disco: 13
- Gestor de almacenamiento: 40, 42, 43
- Índice: 16, 20
- Nivel lógico: 13
- Nivel físico: 13
- SGBD relacional: 4, 7